

CST-407 Milestone 4 Demonstrate and Mitigate SQL Injection Vulnerability

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

09/10/2025

Table of Contents

Step 1 — Baseline: Launch App and Locate the Vulnerable Search.....	3
Database Verified	3
Application Started	4
Sample Events Created	5
Search Page Loaded	6
Vulnerable SQL Query	7
Step 2 — Demonstrate the SQL Injection (proof-of-vulnerability)	8
Baseline Search Works.....	8
Injection Returns All Events	9
App Error from Malformed Query.....	10
Console Stack Trace	11
Step 3 — Fix the Vulnerability (convert to a parameterized query)	12
Code Updated to Use Parameters	12
Application Rebuilt Successfully.....	13
Step 4 — Verify the Fix (prove injection no longer works).....	14
Normal Search Works After Fix	14
Injection No Longer Works.....	15
No Error on Special Characters.....	16
List of Changes (What I Changed and Why).....	17
Summary of Key Points	18

Step 1 — Baseline: Launch App and Locate the Vulnerable Search Database Verified

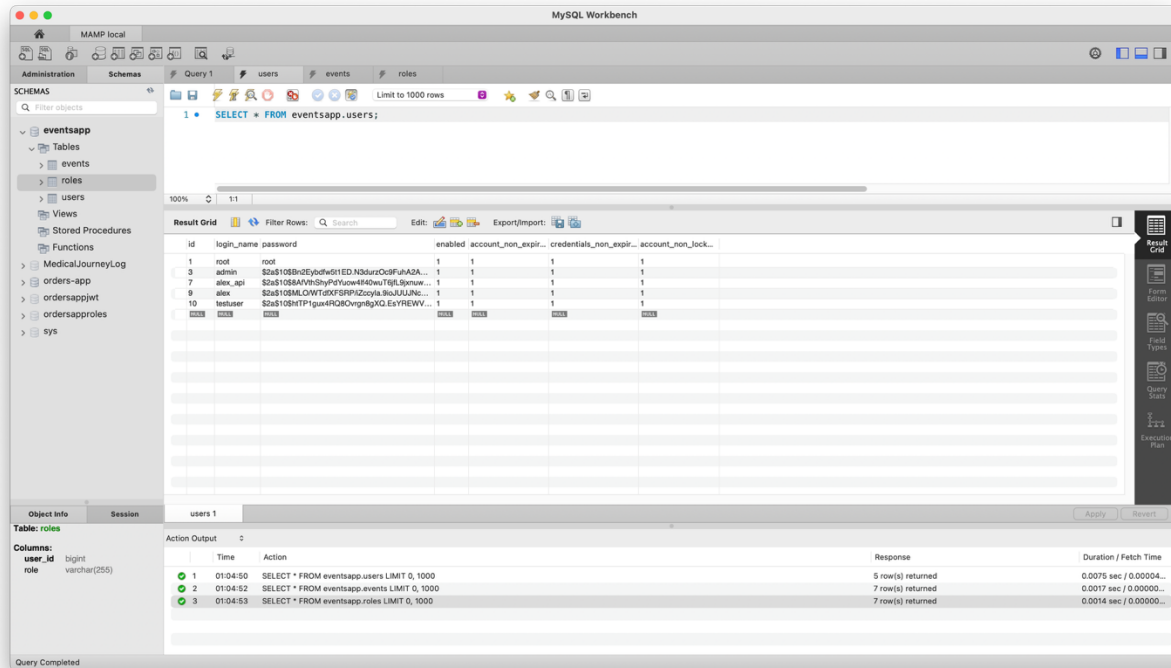


Figure 1 This screenshot confirms that the `eventsapp` schema exists and is ready for the application.

Application Started

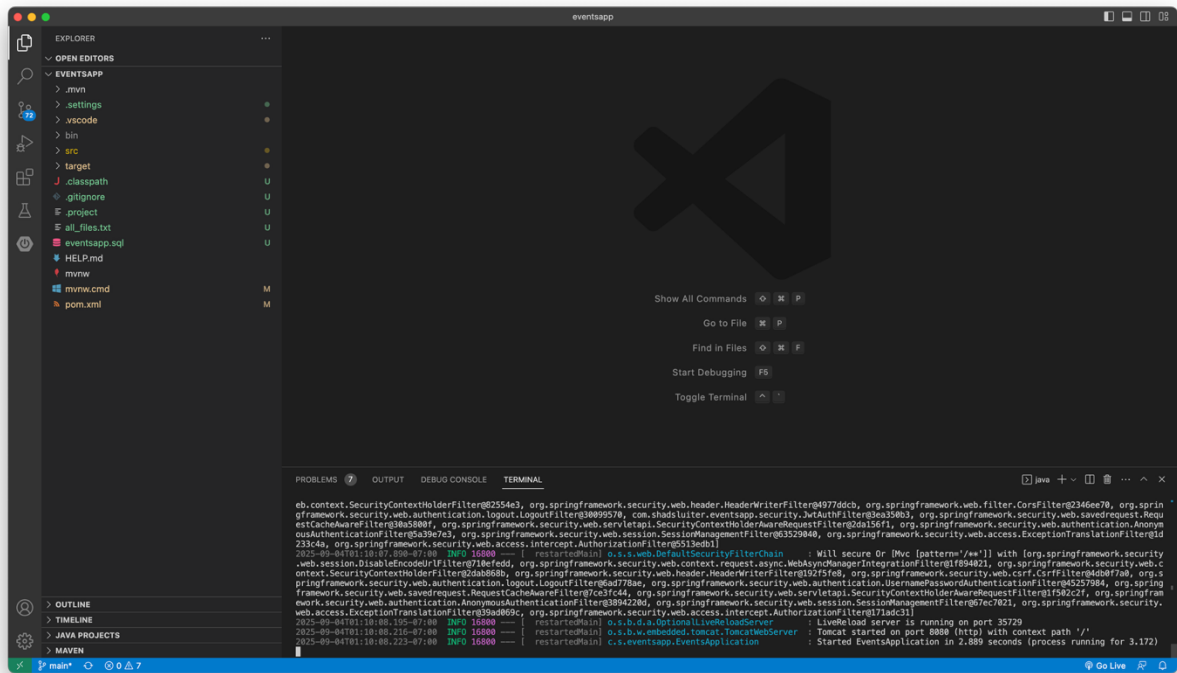
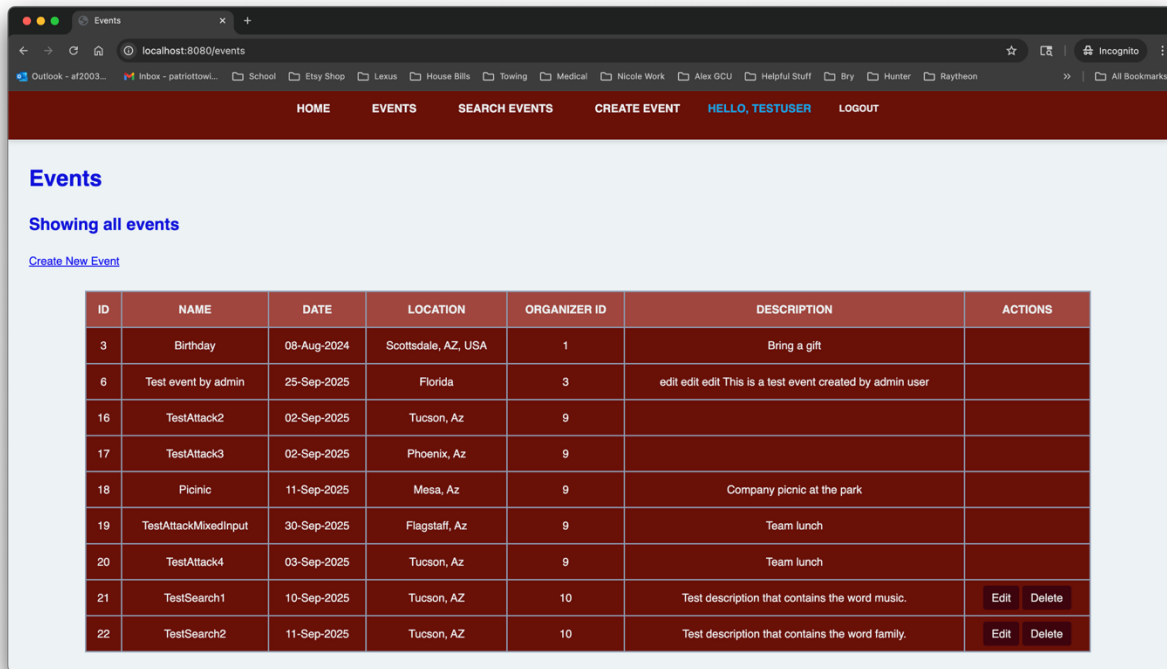


Figure 2 This screenshot shows the console output confirming that the Spring Boot application started successfully on port 8080.

Sample Events Created



ID	NAME	DATE	LOCATION	ORGANIZER ID	DESCRIPTION	ACTIONS
3	Birthday	08-Aug-2024	Scottsdale, AZ, USA	1	Bring a gift	
6	Test event by admin	25-Sep-2025	Florida	3	edit edit edit This is a test event created by admin user	
16	TestAttack2	02-Sep-2025	Tucson, Az	9		
17	TestAttack3	02-Sep-2025	Phoenix, Az	9		
18	Picnic	11-Sep-2025	Mesa, Az	9	Company picnic at the park	
19	TestAttackMixedInput	30-Sep-2025	Flagstaff, Az	9	Team lunch	
20	TestAttack4	03-Sep-2025	Tucson, Az	9	Team lunch	
21	TestSearch1	10-Sep-2025	Tucson, AZ	10	Test description that contains the word music.	Edit Delete
22	TestSearch2	11-Sep-2025	Tucson, AZ	10	Test description that contains the word family.	Edit Delete

Figure 3 This screenshot shows the two test events created, which will later be used to demonstrate SQL injection in the search feature.

Search Page Loaded

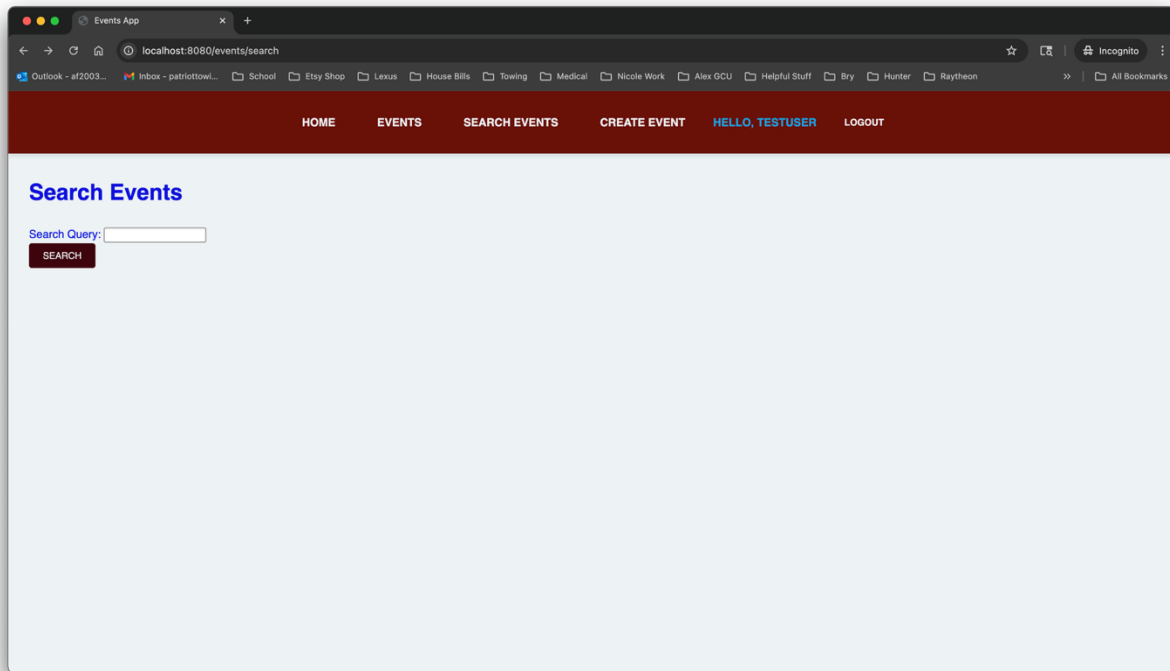


Figure 4 This screenshot confirms that the search feature is accessible at /events/search, where the SQL injection vulnerability will be demonstrated.

Vulnerable SQL Query

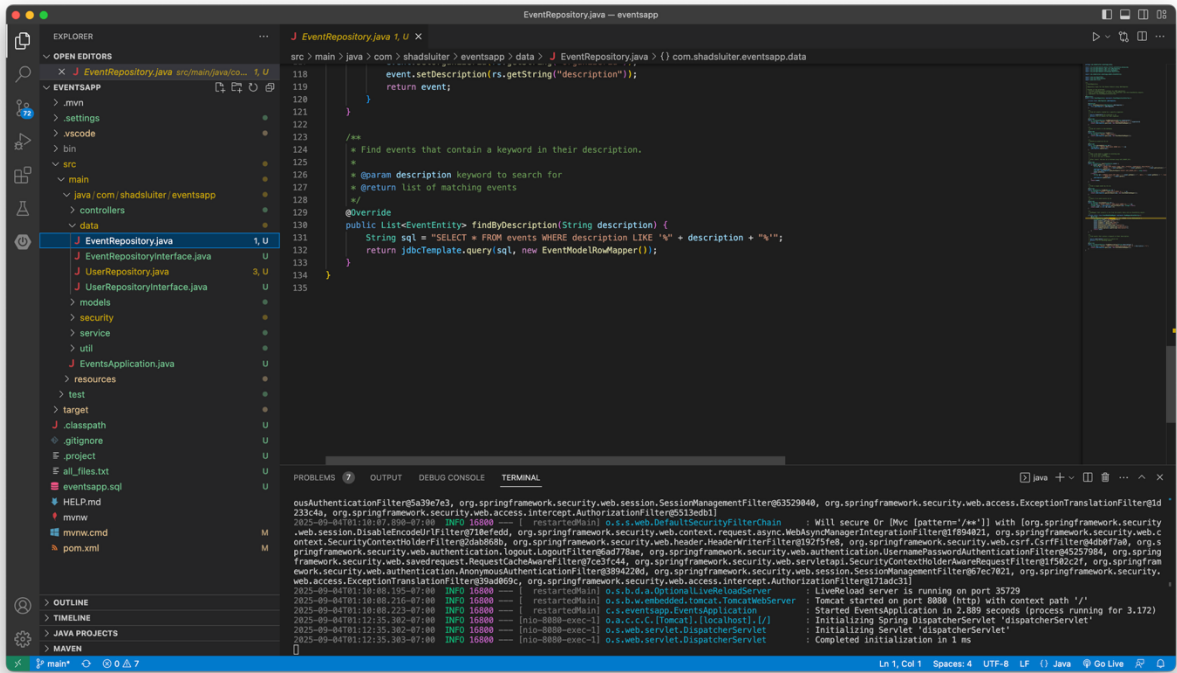


Figure 5 This screenshot highlights the `findByDescription` method in `EventRepository` that concatenates raw user input into the SQL query, creating a SQL injection vulnerability.

Step 2 — Demonstrate the SQL Injection (proof-of-vulnerability)

Baseline Search Works

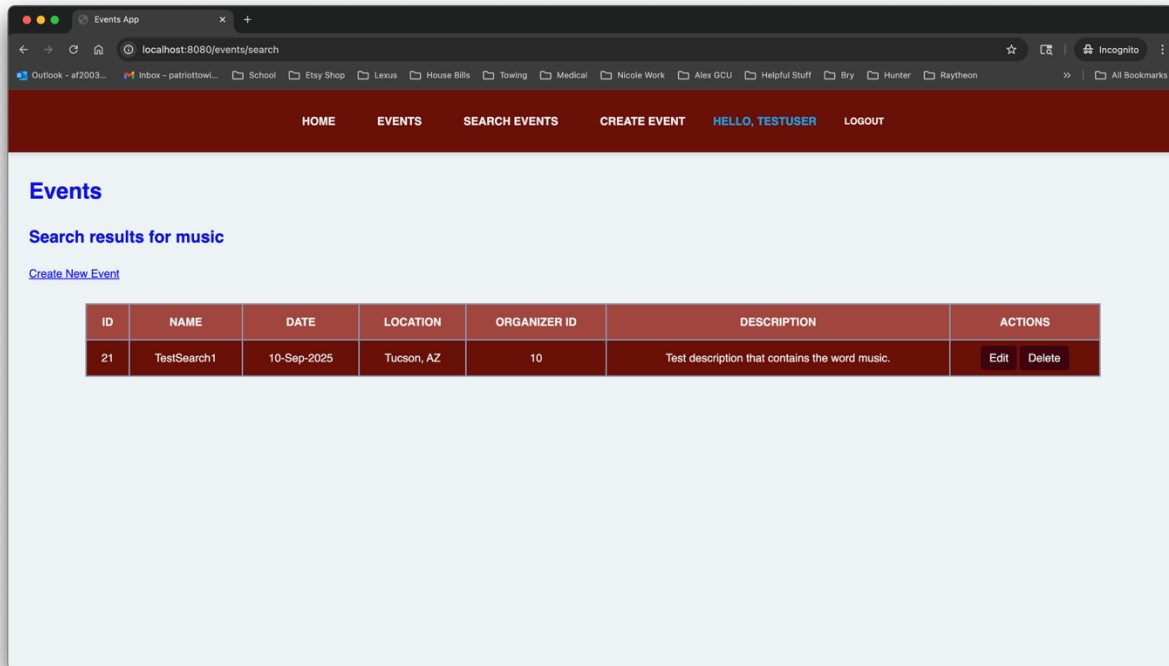


Figure 6 A normal keyword search returns only the matching event, establishing expected behavior before demonstrating injection.

Injection Returns All Events

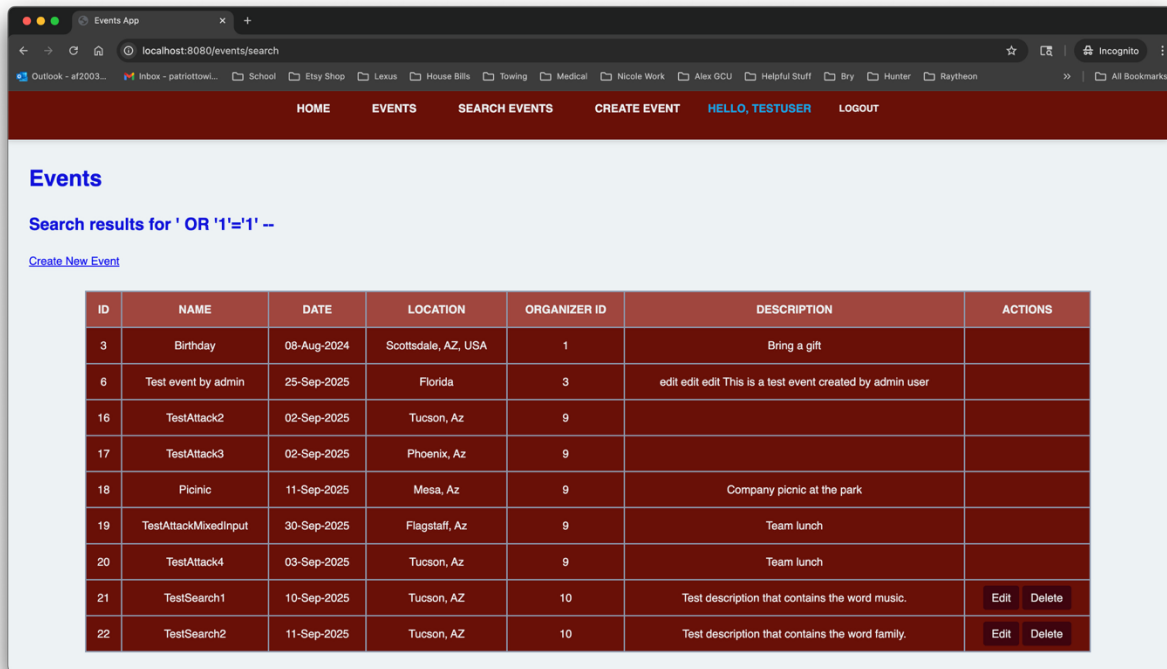


Figure 7 Using the payload ' OR '1'='1' -- forces the WHERE clause to always be true, causing the search to return all events. This proves the input is being injected into SQL.

App Error from Malformed Query

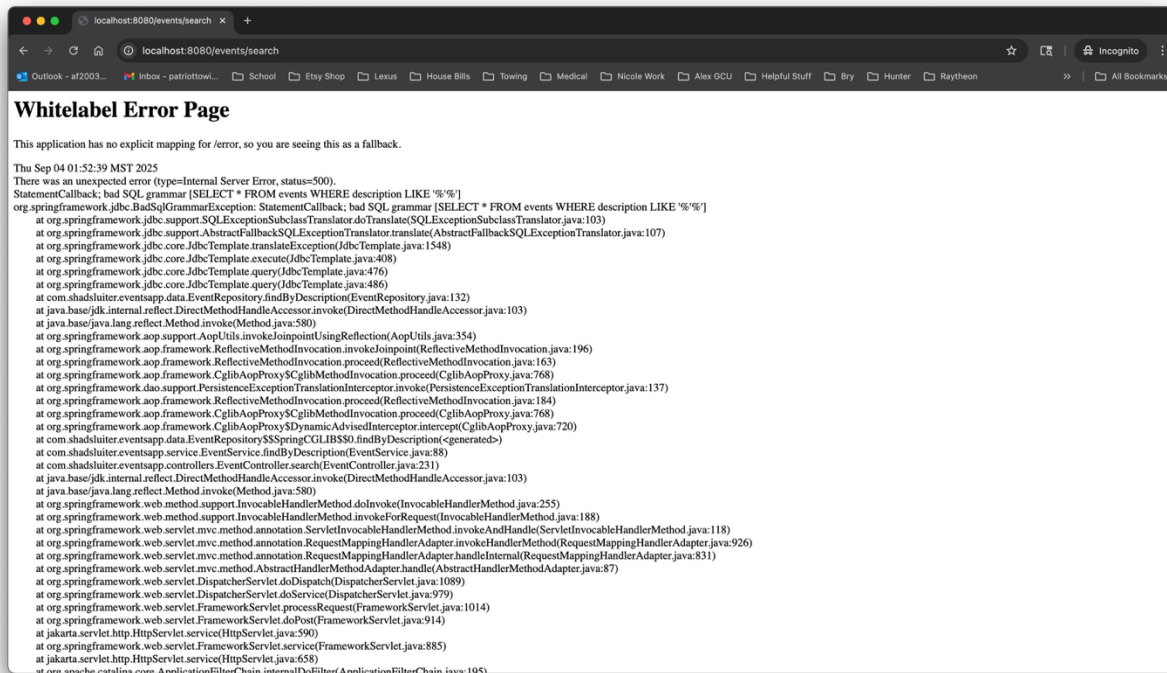
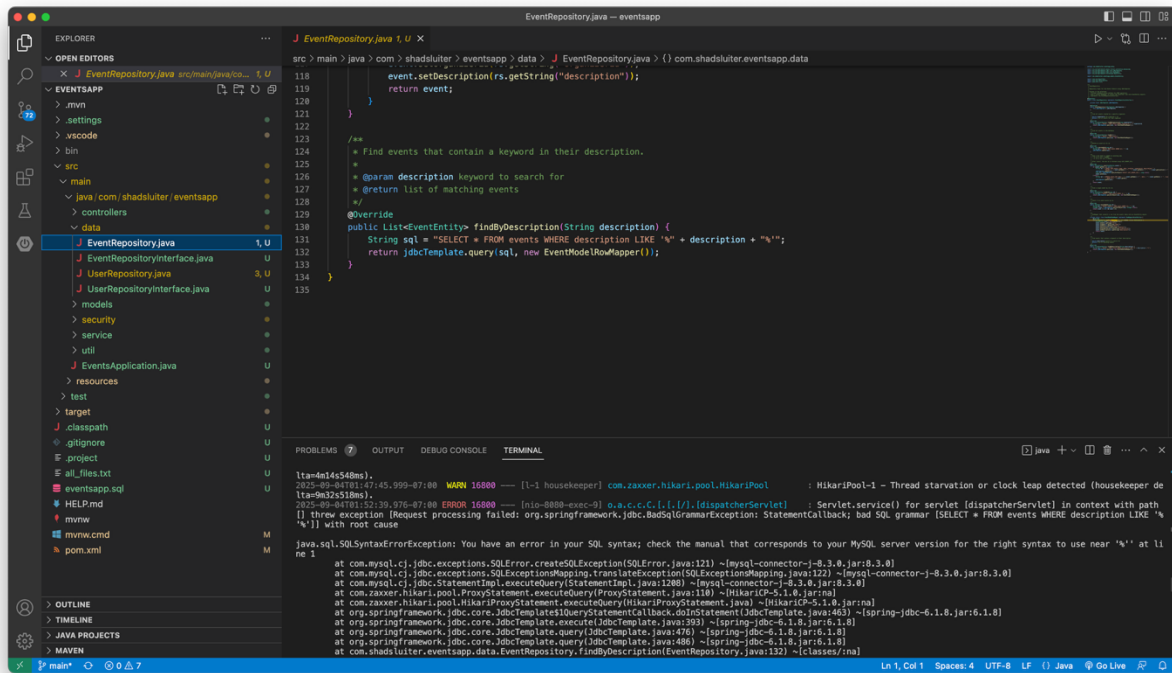


Figure 8 The browser shows an error after submitting a lone quote, demonstrating that unescaped input is directly affecting the SQL statement.

Console Stack Trace



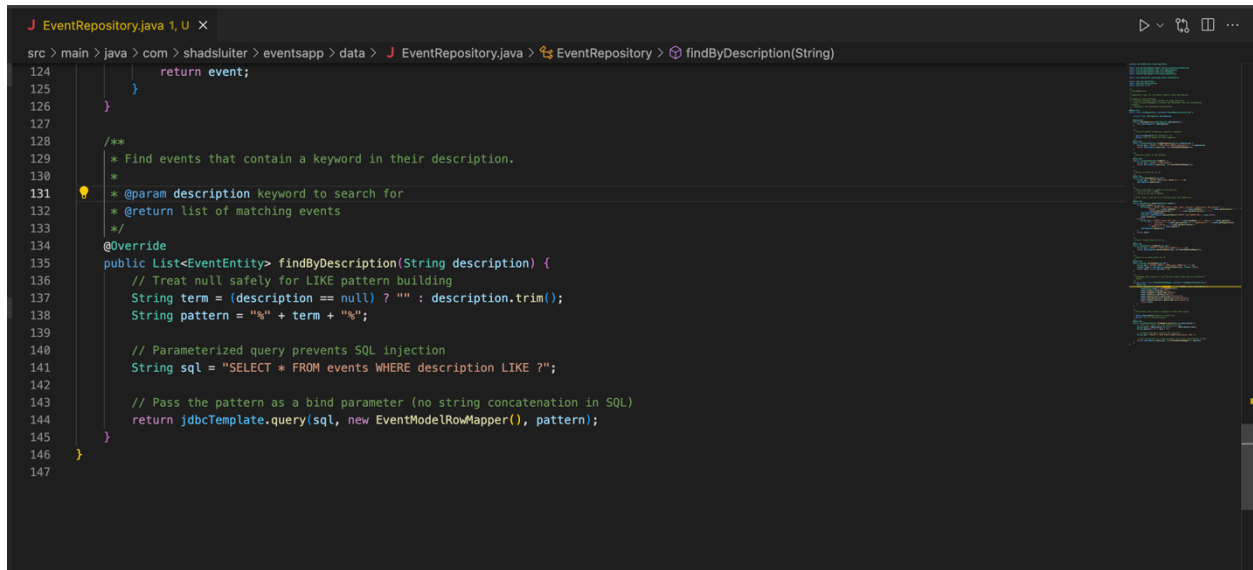
The screenshot shows an IDE with a project named 'eventsapp'. The file 'EventRepository.java' is open, showing a method 'findByDescription' that constructs a SQL query using string concatenation. The query is: `String sql = "SELECT * FROM events WHERE description LIKE '%" + description + "%'";`. The stack trace in the console indicates a `java.sql.SQLException: You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '%'' at line 1`. The stack trace includes the following frames:

```
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122) ~[mysql-connector-j-8.3.0.jar:8.3.0]
at com.mysql.cj.jdbc.StatementImpl.executeQuery(StatementImpl.java:1280) ~[mysql-connector-j-8.3.0.jar:8.3.0]
at com.zaxxer.hikari.pool.ProxyStatement.executeQuery(ProxyStatement.java:118) ~[HikariCP-5.1.0.jar:na]
at com.zaxxer.hikari.pool.HikariProxyStatement.executeQuery(HikariProxyStatement.java) ~[HikariCP-5.1.0.jar:na]
at org.springframework.jdbc.core.JdbcTemplate$QueryStatementCallback.doInStatement(JdbcTemplate.java:463) ~[spring-jdbc-6.1.8.jar:6.1.8]
at org.springframework.jdbc.core.JdbcTemplate.execute(JdbcTemplate.java:393) ~[spring-jdbc-6.1.8.jar:6.1.8]
at org.springframework.jdbc.core.JdbcTemplate.query(JdbcTemplate.java:476) ~[spring-jdbc-6.1.8.jar:6.1.8]
at org.springframework.jdbc.core.JdbcTemplate.query(JdbcTemplate.java:486) ~[spring-jdbc-6.1.8.jar:6.1.8]
at com.shadsluter.eventsapp.data.EventRepository.findByDescription(EventRepository.java:132) ~[classes:/na]
```

Figure 9 The console shows a SQL syntax exception caused by the injected quote, further confirming vulnerable concatenation.

Step 3 — Fix the Vulnerability (convert to a parameterized query)

Code Updated to Use Parameters

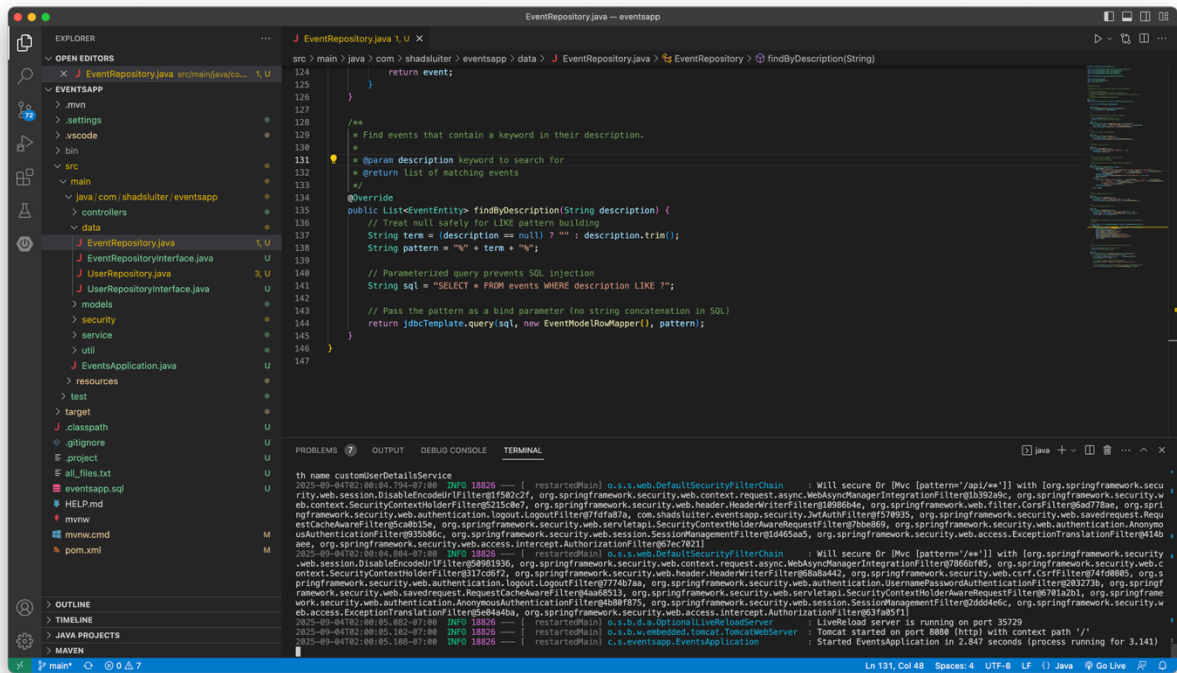


```
J EventRepository.java 1, U X
src > main > java > com > shadsluiter > eventsapp > data > J EventRepository.java > EventRepository > findByDescription(String)

124         return event;
125     }
126 }
127
128 /**
129  * Find events that contain a keyword in their description.
130  *
131  * @param description keyword to search for
132  * @return list of matching events
133  */
134 @Override
135 public List<EventEntity> findByDescription(String description) {
136     // Treat null safely for LIKE pattern building
137     String term = (description == null) ? "" : description.trim();
138     String pattern = "%" + term + "%";
139
140     // Parameterized query prevents SQL injection
141     String sql = "SELECT * FROM events WHERE description LIKE ?";
142
143     // Pass the pattern as a bind parameter (no string concatenation in SQL)
144     return jdbcTemplate.query(sql, new EventModelRowMapper(), pattern);
145 }
146
147
```

Figure 10 This screenshot shows the `findByDescription` method rewritten to use a parameterized `LIKE ?` query, removing the SQL injection risk from string concatenation.

Application Rebuilt Successfully



The screenshot displays an IDE window with the file `EventRepository.java` open. The file contains a `findByDescription(String)` method that constructs a SQL query to find events by description. The console output at the bottom shows the application starting successfully on port 8080.

```
src > main > java > com > shadsluter > eventapp > data > J EventRepository.java > EventRepository > findByDescription(String)
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147

/**
 * Find events that contain a keyword in their description.
 *
 * @param description keyword to search for
 * @return list of matching events
 */
@Override
public List<EventEntity> findByDescription(String description) {
    // Treat null safely for LIKE pattern building
    String term = (description == null) ? "" : description.trim();
    String pattern = "%" + term + "%";

    // Parameterized query prevents SQL injection
    String sql = "SELECT * FROM events WHERE description LIKE ?";

    // Pass the pattern as a bind parameter (no string concatenation in SQL)
    return jdbcTemplate.query(sql, new EventModeRowMapper(), pattern);
}
```

```
th name customUserDetailsService
2025-09-04T02:00:04.706-07:00 INFO 18826 --- [ restartedMain] o.s.s.web.DefaultSecurityFilterChain : Will secure Dr [Mvc [pattern="/api/**]] with [org.springframework.security.web.session.DisableEncodeUrlFilter@f582c2f, org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@b392a9c, org.springframework.security.web.context.SecurityContextHolderFilter@215de7, org.springframework.security.web.header.HeaderWriterFilter@b98b4e, org.springframework.web.filter.CorsFilter@6ad7bae, org.springframework.security.web.authentication.logout.LogoutFilter@7f6f87a, com.shadsluter.eventapp.security.WebAuthFilter@739b35, org.springframework.security.web.savedrequest.RequestCacheAwareFilter@c4b15e, org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@7bbe869, org.springframework.security.web.authentication.AnonymousAuthenticationFilter@923b6c, org.springframework.security.web.session.SessionManagementFilter@1d455a5, org.springframework.security.web.access.ExceptionTranslationFilter@414bae, org.springframework.security.web.access.intercept.AuthorizationFilter@ec7821]
2025-09-04T02:00:04.871-07:00 INFO 18826 --- [ restartedMain] o.s.s.web.DefaultSecurityFilterChain : Will secure Dr [Mvc [pattern="/**]] with [org.springframework.security.web.session.DisableEncodeUrlFilter@5981336, org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@786da4b, org.springframework.security.web.context.SecurityContextHolderFilter@317c0f2, org.springframework.security.web.header.HeaderWriterFilter@6b8a442, org.springframework.security.web.csrf.CsrfFilter@74f0885, org.springframework.security.web.authentication.logout.LogoutFilter@777d78a, org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter@282723b, org.springframework.security.web.savedrequest.RequestCacheAwareFilter@4a8a0513, org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@6781a201, org.springframework.security.web.authentication.AnonymousAuthenticationFilter@4b8f875, org.springframework.security.web.session.SessionManagementFilter@2dd4de6, org.springframework.security.web.access.ExceptionTranslationFilter@544440b, org.springframework.security.web.access.intercept.AuthorizationFilter@63f4b5f]
2025-09-04T02:00:05.082-07:00 INFO 18826 --- [ restartedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-09-04T02:00:05.182-07:00 INFO 18826 --- [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-09-04T02:00:05.188-07:00 INFO 18826 --- [ restartedMain] c.s.eventapp.EventApplication : Started EventsApplication in 2.847 seconds (process running for 3.141)
```

Figure 11 The console output confirms the project compiles and the Spring Boot application starts on port 8080 after the security fix.

Step 4 — Verify the Fix (prove injection no longer works)

Normal Search Works After Fix

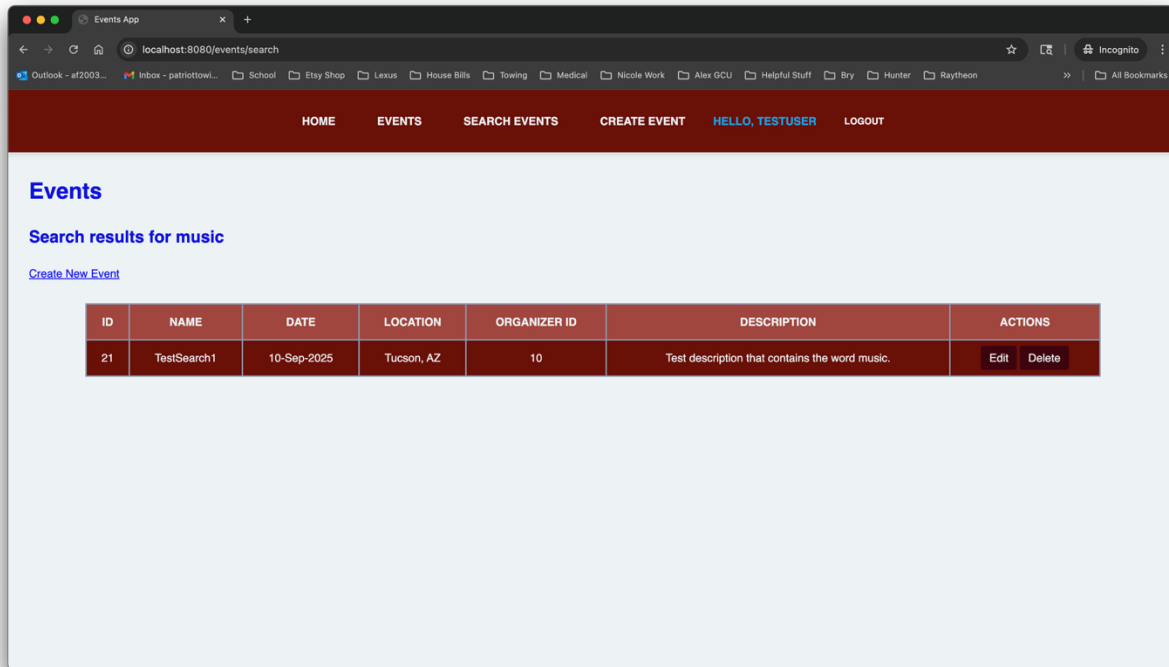


Figure 12 A regular keyword search still returns the expected single match, confirming functionality was preserved after parameterizing the query.

Injection No Longer Works

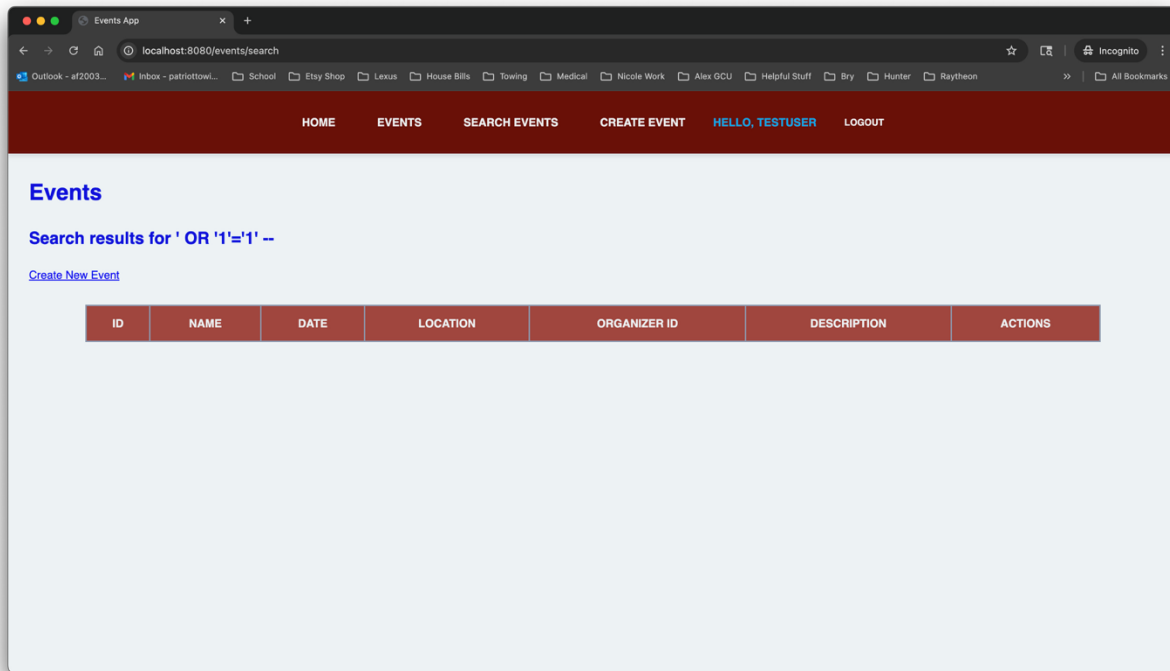


Figure 13 The classic ' OR '1'='1' -- payload no longer alters the query logic and returns no unintended results, proving the injection is mitigated by prepared statements.

No Error on Special Characters

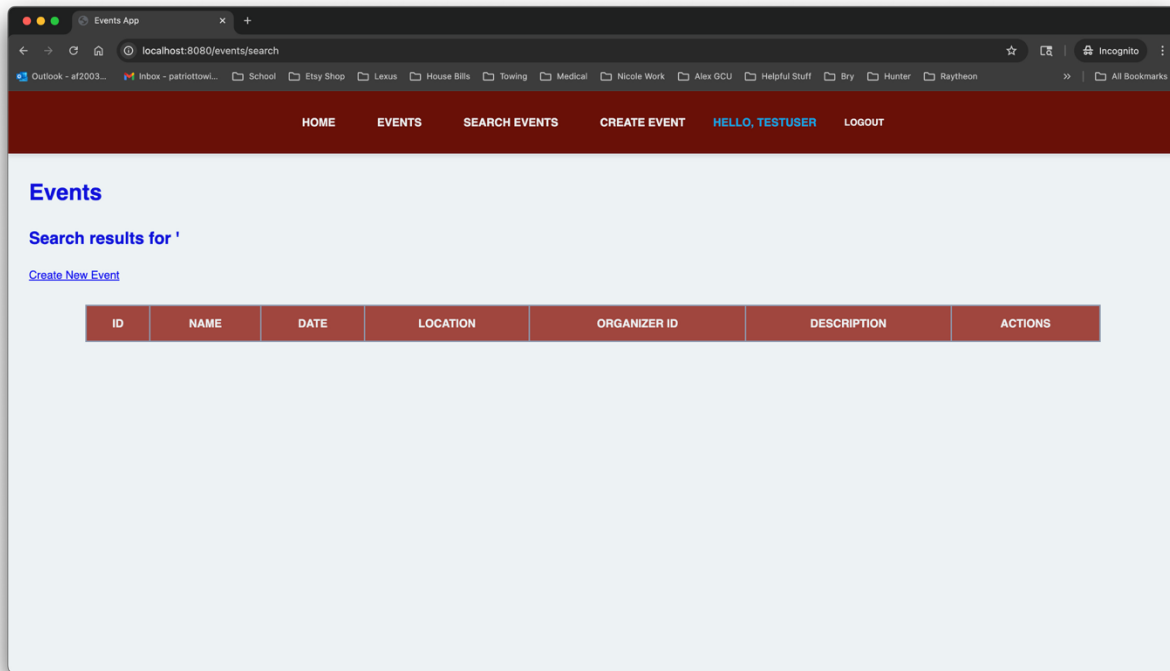


Figure 14 Submitting a single quote does not crash the app, indicating the driver is safely handling the value through bind parameters.

List of Changes (What I Changed and Why)

- Demonstrated SQL injection at /events/search using payload ' OR '1'='1' -- to alter query behavior (returned all rows).
- Reproduced error-based injection with a lone ' to show unescaped input reaching SQL.
- Identified vulnerable code in EventRepository.findByDescription that concatenated user input into the SQL LIKE clause.
- Refactored findByDescription to parameterized SQL: `SELECT * FROM events WHERE description LIKE ?` with a bound pattern ("`%`" + term + "`%`") to eliminate injection.
- Verified fix: normal search still works; boolean-true payload returns 0 results; single quote no longer throws SQL syntax errors.

Summary of Key Points

In Milestone 4, I proved that the Events app's search feature was vulnerable to SQL injection and then fixed it. I used the real UI at **/events/search** and submitted the payload **' OR '1'='1' --** which returned all events, and a lone **'** which triggered an error—clear evidence that user input was being concatenated directly into SQL. I traced the issue to

`EventRepository.findByDescription`, which built the query with "... WHERE description LIKE '%" + description + "%'". I refactored that method to a **parameterized query**:

`SELECT * FROM events WHERE description LIKE ?` and bound the value as `"%" + term + "%"`, removing string concatenation entirely. After restarting the app, normal keyword searches still worked (no feature regression), the boolean-true payload no longer returned all rows (usually 0 results), and a single quote didn't break anything. This confirms the vulnerability is mitigated using prepared statements. My environment was Spring Boot on port 8080 with MySQL at 127.0.0.1:8889 (eventsapp, root/root), so the steps are reproducible.